

AdaBoost Step by Step

Contents

1. Overview	2
2. The toy data	3
3. Step 1: every row gets the same weight	3
4. Step 2: fit a decision stump	3
5. Step 3: the error is a sum of weights	4
6. Step 4: the say of the stump, alpha	4
6.1 What shape alpha should have	4
6.2 The formula	5
7. Step 5: update the row weights	6
8. Step 6: normalise the weights	6
9. Step 7: upsampling, a new dataset drawn by weight	7
10. Repeat, then vote	8
11. Summary	9
12. Sources	9
13. Key terms	9

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Decision trees ([Note 100](#)); Boosting ([Note 101](#)); Hyperparameter tuning ([Note 111](#)).
- **Compare with:** Gradient boosting ([Note 120](#)).

1. Overview

Key point: One AdaBoost stage, in seven steps: give the rows weights, fit a stump, add up the weights it got wrong, turn that error into a say alpha, raise the weights of the mistakes and lower the rest, normalise, and draw a new dataset by weight.

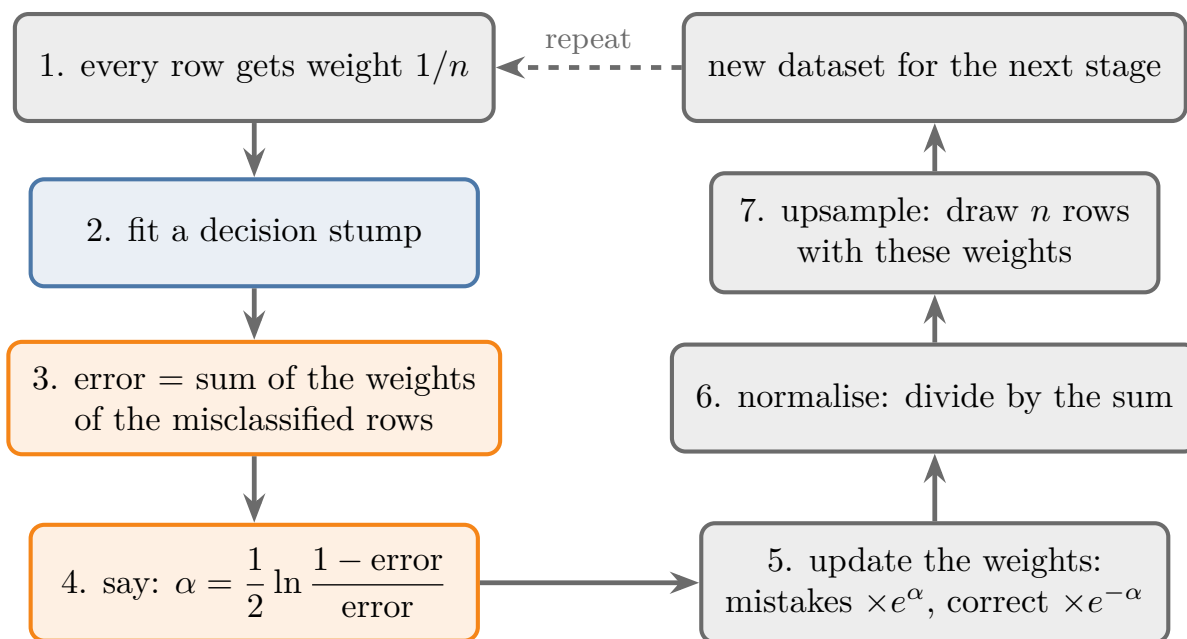


Figure 1: One stage of AdaBoost; the new dataset starts the next stage

We met the idea in the [AdaBoost intuition Note](#): stumps trained one after another, each focusing on the previous one's mistakes, combined by a weighted vote. It left two questions open:

- how the say α of each stump is computed;
- how the mistakes are made “more important” for the next stump.

This Note answers both on a toy dataset of 5 rows, following Figure 1 step by step. The [AdaBoost from scratch Note](#) runs the same steps in Python.

2. The toy data

Key point: 5 rows, two input columns x1 and x2, and a class y of +1 or -1.

Row	x1	x2	y
1	3	7	+1
2	2	9	+1
3	1	9	-1
4	9	8	-1
5	7	4	-1

The output has two classes, so this is classification. The number of rows is $n = 5$. As in the intuition Note, the classes are written +1 and -1.

3. Step 1: every row gets the same weight

Key point: At the start every row has weight $1/n$, so all rows are equally important and the weights add up to 1.

AdaBoost gives each row a **sample weight**: a number saying how important that row is. At the start all rows are equally important.

1. **In words:** each row's starting weight is 1 divided by the number of rows.
2. **Formula:**

$$w_i = \frac{1}{n}$$

3. **Example:** with $n = 5$, every row gets $w_i = 1/5 = 0.2$, and $0.2 \times 5 = 1$.

The weights always add up to 1. We add them to the table as a new column, *weight*.

4. Step 2: fit a decision stump

Key point: Train a decision stump on the data and record its prediction for every row.

We train a decision stump: a decision tree with `max_depth=1` (the intuition Note, section 2.2). It tries every cut on x1 and on x2, for example “ $x1 > 5$ ” or “ $x2 < 10$ ”, and keeps the one with the largest information gain (or the largest drop in Gini impurity: the [decision trees Note](#) compares the two). Call the result **model 1**.

We then pass the training rows through model 1 and write its predictions in a new column. Suppose they are:

Row	y	weight	prediction	correct?
1	+1	0.2	+1	yes

Row	y	weight	prediction	correct?
2	+1	0.2	-1	no
3	-1	0.2	+1	no
4	-1	0.2	-1	yes
5	-1	0.2	-1	yes

Model 1 gets rows 2 and 3 wrong. The predictions are an assumed example: the steps that follow are the same for any stump.

5. Step 3: the error is a sum of weights

Key point: A stump's error is the total weight of the rows it misclassified, not simply the share of rows it got wrong.

1. **In words:** add up the weights of the misclassified rows.
2. **Formula:**

$$\text{error} = \sum_{i: \hat{y}_i \neq y_i} w_i$$

The sum runs over the rows where the prediction $\hat{y}_i$ differs from the true class y_i .

3. **Example:** rows 2 and 3 are wrong, each with weight 0.2:

$$\text{error} = 0.2 + 0.2 = 0.4$$

In this first stage all weights are equal, so the error equals the share of rows wrong, 2 out of 5. In later stages the weights differ, and a mistake on a heavy row costs more than a mistake on a light one.

6. Step 4: the say of the stump, alpha

Key point: Alpha, half the log of $(1 - \text{error})/\text{error}$, is large and positive for a small error, 0 at error 0.5, and large and negative for an error near 1.

The **model weight** α is the stump's say in the final vote. It must depend on the error: a model that makes many mistakes should have a small say, and a model that makes few a large one.

6.1 What shape alpha should have

Key point: A model that is always wrong is as useful as one that is always right: we just flip its answer. The useless model is the one that is right half the time.

Take three models:

- **model A** is always right: error 0, accuracy 100%;
- **model B** is always wrong: error 1, accuracy 0%;

- **model C** is right half the time: error 0.5, accuracy 50%.

(The error is 1 minus the accuracy: 90% accuracy means an error of 0.1.)

A is clearly the most reliable. Between B and C, B is the more reliable. Think of two friends: one lies every time, the other lies half the time. Whatever the first one says, the opposite is true, so we can trust that friend completely by flipping every answer. The second one is no help at all: we never know when to believe them.

So we want a function of the error that:

- is large and positive when the error is near 0 (trust the model);
- is 0 when the error is 0.5 (ignore the model);
- is large and negative when the error is near 1 (trust the model, but flip its vote).

6.2 The formula

Key point: Half the natural log of the ratio of correct weight to wrong weight.

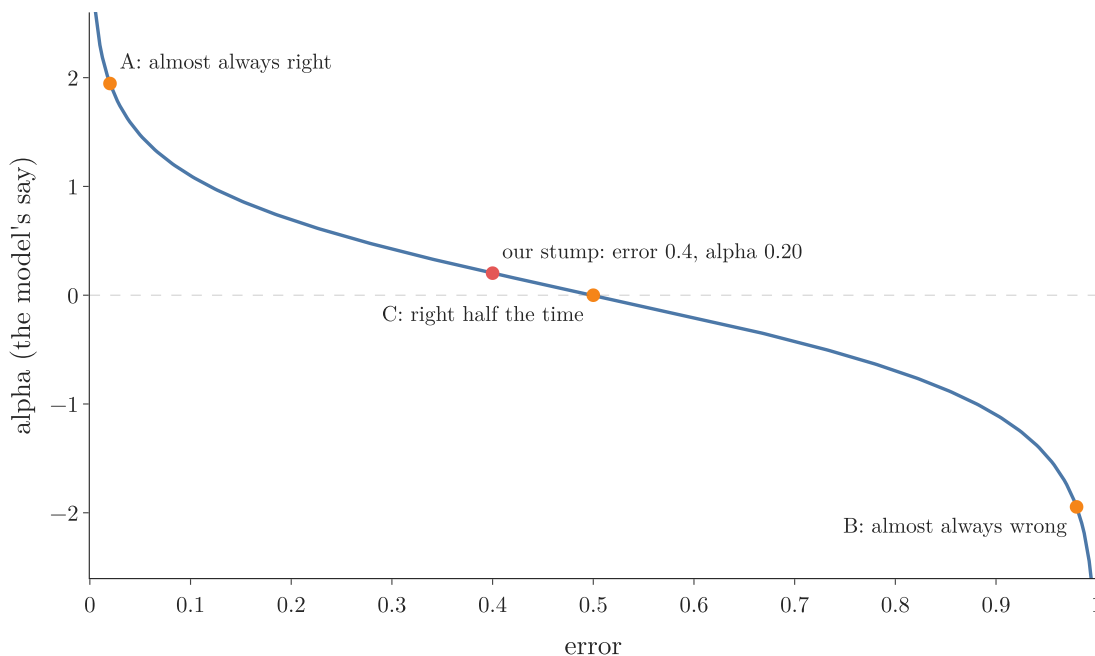


Figure 2: Alpha against the error: model A (error near 0) gets a large positive say, model C (error 0.5) none, model B (error near 1) a large negative say

The function in Figure 2 has exactly that shape.

1. **In words:** divide the weight the stump got right by the weight it got wrong, take the natural logarithm, and halve it.
2. **Formula:**

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \text{error}}{\text{error}} \right)$$

$\ln$ is the natural logarithm, the log to base $e \approx 2.718$ (logs were introduced in the [log loss Note](#), section 4).

3. **Example:** model 1 has error 0.4:

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - 0.4}{0.4} \right) = \frac{1}{2} \ln(1.5) = \frac{1}{2} \times 0.405 = 0.20$$

So model 1's say in the final vote is $\alpha_1 = 0.20$, a small say, since it got 40% wrong (Figure 2, red point). The checks: an error of 0.5 gives $0.5 \times \ln 1 = 0$, and an error of 0.98 gives $0.5 \times \ln(0.02/0.98) = -1.95$, the mirror image of an error of 0.02.

Extra: A negative alpha flips the stump's vote in the final sum, which is the “believe the liar backwards” idea. In practice a stump that is worse than guessing is rarely kept: scikit-learn stops adding stumps when a new one's error reaches 0.5 or more on two classes (scikit-learn source; the [AdaBoost hyperparameters Note](#)).

7. Step 5: update the row weights

Key point: Multiply the weight of each misclassified row by e^α (it grows) and of each correct row by $e^{-\alpha}$ (it shrinks).

Now we tell the next stump about the mistakes, by **boosting** the weights of the misclassified rows and lowering the rest. This is where boosting gets its name.

1. **In words:** a misclassified row's weight is multiplied by e to the power alpha; a correctly classified row's weight by e to the power minus alpha.
2. **Formula:**

$$w_i^{\text{new}} = \begin{cases} w_i e^\alpha & \text{if row } i \text{ was misclassified} \\ w_i e^{-\alpha} & \text{if row } i \text{ was classified correctly} \end{cases}$$

3. **Example:** with $w_i = 0.2$ and $\alpha_1 = 0.2027$:

$$\text{misclassified: } 0.2 \times e^{0.2027} = 0.2 \times 1.2247 = 0.2449$$

$$\text{correct: } 0.2 \times e^{-0.2027} = 0.2 \times 0.8165 = 0.1633$$

Rows 2 and 3 rise from 0.2 to about 0.24; rows 1, 4 and 5 fall to about 0.16. Why the exponential is the right choice is shown in the [AdaBoost from scratch Note](#).

Extra: With classes +1 and -1, both cases fit in one formula: $w_i^{\text{new}} = w_i e^{-\alpha y_i h(x_i)}$. If the stump is right, $y_i h(x_i) = (+1)(+1)$ or $(-1)(-1) = +1$, giving $e^{-\alpha}$. If it is wrong, $y_i h(x_i) = -1$, giving e^α . This is another reason AdaBoost uses +1 and -1.

8. Step 6: normalise the weights

Key point: Divide every new weight by their total, so the weights add up to 1 again.

After the update the weights no longer add up to 1.

1. **In words:** divide each weight by the sum of all the weights.
2. **Formula:**

$$w_i \leftarrow \frac{w_i^{\text{new}}}{\sum_{j=1}^n w_j^{\text{new}}}$$

3. **Example:** the sum is $2 \times 0.2449 + 3 \times 0.1633 = 0.4899 + 0.4899 = 0.9798$, so

$$\text{misclassified: } \frac{0.2449}{0.9798} = 0.25 \quad \text{correct: } \frac{0.1633}{0.9798} = 0.1667$$

Check: $2 \times 0.25 + 3 \times 0.1667 = 1$. The two mistakes now carry half of the total weight between them, against 40% before.

Extra: Here the misclassified rows end up with exactly half the total weight. That is always so after a normalised AdaBoost update. Before normalising, the mistakes weigh $\text{error} \cdot e^\alpha$ in total and the correct rows $(1 - \text{error}) \cdot e^{-\alpha}$. With $e^\alpha = \sqrt{(1 - \text{error})/\text{error}}$ from step 4, both totals equal $\sqrt{\text{error}(1 - \text{error})}$: in our example $\sqrt{0.4 \times 0.6} = 0.4899$, the two equal halves of the sum above. So the old stump, judged on the new weights, has an error of exactly 0.5 and would get $\alpha = 0$: repeating it adds nothing to the vote, and the next stump only earns a say by doing better on the reweighted rows.

9. Step 7: upsampling, a new dataset drawn by weight

Key point: Lay the weights end to end on the line from 0 to 1, draw n random numbers, and pick the row whose range each number lands in. Heavy rows get picked more often.

The new weights are passed on through the data itself. We build a new dataset of the same size, $n = 5$ rows, in which heavy rows appear more often. This is called **upsampling** (resampling by weight).

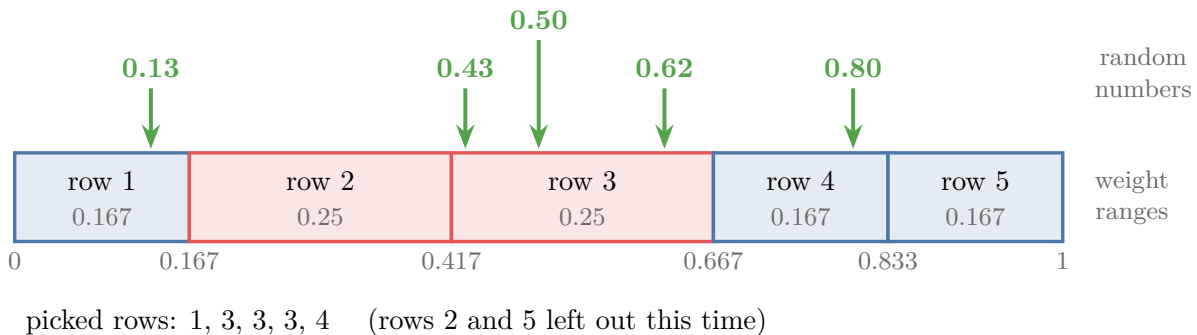


Figure 3: Upsampling: each row owns a stretch of the line from 0 to 1 as long as its weight; five random numbers pick rows 1, 3, 3, 3 and 4

1. **Make ranges.** Each row owns a stretch of the line from 0 to 1, as long as its weight. Row 1 owns 0 to 0.167, row 2 owns 0.167 to 0.417, row 3 owns 0.417 to 0.667, row 4 owns 0.667 to 0.833, and row 5 owns 0.833 to 1 (Figure 3). Each boundary is the running total, the **cumulative sum**, of the weights.

2. **Draw random numbers.** Draw 5 random numbers between 0 and 1, say 0.13, 0.43, 0.62, 0.50 and 0.80.
3. **Pick rows.** Each number picks the row whose range it falls in: 0.13 picks row 1; 0.43, 0.62 and 0.50 all pick row 3; 0.80 picks row 4.

The new dataset is rows 1, 3, 3, 3 and 4. Row 3, a mistake, appears three times; rows 2 and 5 do not appear at all. Another draw could give, for example, rows 1, 3, 2, 2 and 5. Either way, rows with larger weights own longer stretches of the line and are picked more often.

The next stump trains on this new dataset, so it pays most attention to the rows the first stump got wrong.

Extra: Upsampling is one way to make a model respect weights. The other is to hand the weights straight to the learner: scikit-learn's decision trees accept a `sample_weight` argument and count each row in proportion to its weight. scikit-learn's AdaBoost works this way, with no random draws (scikit-learn source; the [AdaBoost from scratch Note](#), section 10).

10. Repeat, then vote

Key point: The new dataset starts the next stage; after T stages we have T stumps and T alphas, combined by the weighted vote.

With the new dataset, everything repeats (Figure 1):

1. every row of the new dataset gets weight $1/n$ again;
2. a new stump is trained on it, model 2;
3. its error and its say α_2 are computed;
4. the weights are updated and normalised, and a new dataset is drawn for model 3.

We repeat this for as many stumps as we want, T . At the end we have $\alpha_1, \alpha_2, \dots, \alpha_T$ and stumps $h_1, \dots, h_T$, and the prediction is the weighted vote of the intuition Note, section 5:

$$H(x) = \text{sign} \left(\alpha_1 h_1(x) + \alpha_2 h_2(x) + \dots + \alpha_T h_T(x) \right)$$

11. Summary

Step	What we do	Our numbers (stage 1)
1	weight every row $1/n$	0.2 each
2	fit a decision stump	rows 2 and 3 wrong
3	error = sum of the weights of the mistakes	0.4
4	$\alpha = 0.5 \ln((1 - \text{error})/\text{error})$	0.20
5	mistakes $\times e^\alpha$, correct $\times e^{-\alpha}$	0.2449 and 0.1633
6	divide by the sum	0.25 and 0.1667
7	draw n rows by weight	rows 1, 3, 3, 3, 4

- The error is a weighted error: the total weight of the misclassified rows.
- Alpha is large for a small error, 0 at error 0.5, negative above 0.5.
- The weights of the mistakes grow, the others shrink, and all are rescaled to add up to 1.
- Upsampling turns weights into a new dataset in which heavy rows appear more often.

12. Sources

- scikit-learn source: `sklearn/ensemble/_weight_boosting.py`, method `_boost` of `AdaBoostClassifier` (it fits each stump with `sample_weight` and stops when the error reaches $1 - 1/K$, which is 0.5 for two classes).

13. Key terms

Term	Meaning
Sample weight	A number attached to each row saying how important it is; AdaBoost starts every row at $1/n$
Weighted error	The total sample weight of the rows a model misclassifies
Weight update	Multiplying misclassified rows' weights by e^α and correct rows' weights by $e^{-\alpha}$
Normalisation (of weights)	Dividing every weight by their sum so they add up to 1
Upsampling (resampling by weight)	Drawing a new dataset in which each row is picked with probability equal to its weight
Cumulative sum	The running total of a list of numbers; it turns weights into ranges on the line from 0 to 1